

S2

Sémantique SQL – chapitre 1

Jointures : appairer, multiplier, conserver

Pourquoi votre résultat contient des doublons, ou pourquoi une ligne a disparu sans prévenir.

TU ES ICI SI...

- ▶ « Chaque client apparaît trois fois dans mon résultat. »
- ▶ « Un client existe dans la table, mais il n'apparaît pas dans ma jointure. »
- ▶ « J'ai ajouté `DISTINCT` et ça a l'air correct, mais je ne sais pas pourquoi. »
- ▶ « Comment obtenir la liste de ceux qui n'ont rien ? »

Une jointure ne « fusionne » pas deux tables ligne à ligne : elle **apparie**. Le SGBD forme toutes les combinaisons possibles, puis ne garde que celles qui satisfont la condition. Une ligne de gauche peut donc produire **0, 1 ou plusieurs** lignes de résultat – d'où les disparitions et les doublons.

Le mécanisme : combiner, puis filtrer

film (3 lignes)

title	cat
Bucket Brotherhood	1
Frost Head	5
Island Empire	3

category (2 lignes)

id	name
1	Action
5	Horreur

Toutes les combinaisons ($3 \times 2 = 6$), puis le filtre `cat = id`

title	cat	id	name
Bucket Brotherhood	1	1	Action
Bucket Brotherhood	1	5	Horreur
Frost Head	5	1	Action
Frost Head	5	5	Horreur
Island Empire	3	1	Action
Island Empire	3	5	Horreur

La jointure interne conserve les 2 lignes vertes. Island Empire, dont la catégorie 3 n'existe pas dans cet extrait, **disparaît** : aucune combinaison ne le concernant ne passe le filtre.

Interne ou externe : que deviennent les lignes sans partenaire ?

JOIN (interne)

```
SELECT cu.last_name, r.rental_id
FROM   customer cu
       JOIN rental r ON cu.customer_id = r.customer_id;
```

last_name	rental_id
Dupont	1
Dupont	2
Martin	3
Martin	16

4 lignes. Petit, qui n'a jamais rien loué, n'apparaît pas.

LEFT JOIN (externe)

```
SELECT cu.last_name, r.rental_id
FROM   customer cu
       LEFT JOIN rental r ON cu.customer_id = r.
       customer_id;
```

last_name	rental_id
Dupont	1
Dupont	2
Martin	3
Martin	16
Petit	NULL

5 lignes. Petit est conservé, et les colonnes de `rental` sont remplies de **NULL** : c'est une ligne de remplissage.

LEFT JOIN garde toutes les lignes de la table de gauche. **RIGHT JOIN** fait l'inverse (on peut toujours s'en passer en échangeant les deux tables), **FULL JOIN** garde les deux côtés.

POURQUOI VOUS AVEZ DES DOUBLONS

Dupont apparaît deux fois ci-dessus, et c'est **normal** : il a deux locations. La jointure produit une ligne *par location*, pas une ligne par client. Si votre énoncé demande « la liste des clients », la granularité ne correspond pas – il faut **DISTINCT**, ou un regroupement (**GROUP BY**).

DISTINCT : ni bon ni mauvais, cela dépend de la question✓ **SANS DISTINCT – LES RÉPÉTITIONS SONT L'INFORMATION**

« Les acteurs et les films où ils jouent »

```
SELECT a.last_name, f.title
FROM actor a
      JOIN film_actor fa ON a.actor_id = fa.actor_id
      JOIN film f ON fa.film_id = f.film_id;
```

Un acteur doit apparaître autant de fois qu'il a de films. **DISTINCT** serait ici une **erreur**.

✓ **AVEC DISTINCT – LA GRANULARITÉ DEMANDÉE EST LE CLIENT**

« Les clients ayant loué au moins un film d'action »

```
SELECT DISTINCT cu.last_name
FROM customer cu
      JOIN rental r ON cu.customer_id = r.customer_id
      JOIN film f ON r.film_id = f.film_id
      JOIN category c ON f.category_id = c.category_id
WHERE c.name = 'Action';
```

Sans lui, un client apparaît une fois par location.

Le motif « anti-jointure » : ceux qui n'ont rien**Les films jamais loués**

```
SELECT f.title
FROM film f
      LEFT JOIN rental r ON f.film_id = r.film_id
WHERE r.rental_id IS NULL;
```

TESTEZ UNE COLONNE QUI NE PEUT PAS ÊTRE VIDE

On teste `r.rental_id` (la clé primaire), jamais `r.return_date` : celle-ci vaut **NULL** aussi bien pour « aucune location » que pour « location en cours ». On ne saurait plus distinguer les deux cas.

1. le **LEFT JOIN** conserve tous les films; 2. le **WHERE** ne garde que les lignes de remplissage, c'est-à-dire exactement les films sans aucune location.

Sur notre extrait : Academy Dinosaur et Hunting Musketeers.

Deux écritures équivalentes, souvent plus lisibles : `WHERE f.film_id NOT IN (SELECT film_id FROM rental)` et `WHERE NOT EXISTS (SELECT * FROM rental r WHERE r.film_id = f.film_id)`.

LA RÈGLE DE DÉCISION

Deux questions, dans cet ordre, avant d'écrire une jointure :

1. « Une ligne de mon résultat, ce sera quoi ? » (un client ? une location ? un couple acteur-film ?) → cela décide du **DISTINCT** et du regroupement.
2. « Les lignes sans correspondance doivent-elles apparaître ? » → oui : jointure externe ; non : jointure interne.

CE QUE POSTGRESQL VOUS DIT

ERROR: column reference "customer_id" is ambiguous

Deux tables jointes possèdent une colonne de même nom : préfixez-la par l'alias de sa table (`r.customer_id`). C'est précisément à cela que servent les alias.

VÉRIFIE QUE TU AS COMPRIS

1. `customer` contient 3 lignes, `rental` en contient 4. Combien de lignes renvoie `SELECT * FROM customer, rental;` (sans condition) ?
2. Vous joignez `film` et `rental` en interne, et obtenez 4 lignes alors que `film` en compte 6. Que s'est-il passé ?
3. Un **LEFT JOIN** peut-il renvoyer *moins* de lignes que la table de gauche ?
4. Vous voulez « les clients n'ayant jamais loué de film d'horreur ». Le motif **LEFT JOIN** + `IS NULL` suffit-il tel quel ?

RÉPONSES

1. $3 \times 4 = 12$: c'est le produit cartésien, toutes les combinaisons.
2. Deux films n'ont aucune location : la jointure interne les a écartés. Avec un **LEFT JOIN** vous en auriez 6.
3. Non, jamais moins – mais il peut en renvoyer *plus*, si une ligne de gauche a plusieurs correspondances.
4. Non : ajouter `WHERE ca.name = 'Horreur'` annulerait l'effet externe. Il faut soit filtrer dans le **ON**, soit utiliser `NOT EXISTS`.